Microservice Anti-Patterns

all the costs, none of the benefits

Watch the 4-min video

In one line: most microservice pain comes from a few anti-patterns (the distributed monolith, shared databases, chatty calls, long synchronous chains, nano-services and unowned services) and the fix is the same: boundaries around business capabilities, private data, events over chains, and a team for every service.

1 Thirty services, five teams per release

A team splits its monolith into thirty services. A year later every release needs five teams to deploy together, one slow service brings down checkout, and nobody knows who owns half the code. They have the costs of microservices (lesson 19) and none of the benefits.



Figure 1. Six anti-patterns. The highlighted two are the most common causes of outages and release pain.

2 The anti-patterns

Distributed monolith. Separate processes, but tightly coupled: they must deploy together, a change in one forces changes in others, and they share libraries containing business logic. You get network latency and partial failure plus monolith coupling.

Shared database. Several services read and write the same tables, so no one can change a schema safely and one service's heavy query slows the rest (lesson 23). If two services must always change their schema together, they are really one service.

Chatty communication. One page needs dozens of calls between services, usually a sign that boundaries cut through one capability.

Long synchronous chains. Ten hops deep, latencies add and availabilities multiply: $0.999^{10} \approx 99.0\%$. One slow hop makes every caller wait until timeouts cascade (lessons 4 and 6).

Nano-services and missing ownership. Services of one function each carry more deployment and monitoring overhead than logic. Services with no team, no on-call and no approver rot. Test: if you can't explain a service in one sentence of business language, its boundary is probably wrong.

3 Symptoms and fixes

Symptom	Likely anti-pattern	Fix
releases need several services in lock-step	distributed monolith	redraw boundaries by bounded context (lesson 22); version APIs
schema change breaks another team	shared database	database per service; share via API or events
p99 latency explodes when one service slows	long sync chain	events for side effects, local caches, timeouts, circuit breakers
a feature touches 4+ services	wrong or too-fine boundaries	merge services that always change together
alerts nobody answers	no ownership	an owning team and on-call for every service

Sometimes the right fix is to merge back into a **modular monolith** (lesson 20). Measure progress with deployment frequency per service and the share of changes that touch more than one service.

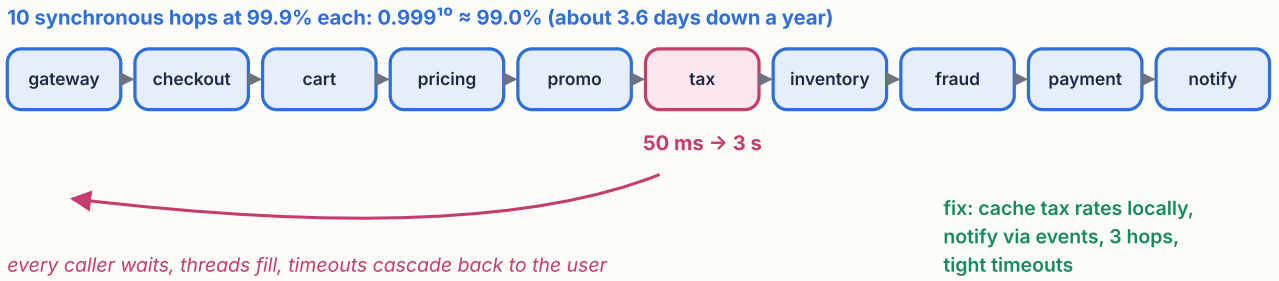


Figure 2. The worked example: one slow hop deep in a synchronous chain takes down checkout.

WORKED EXAMPLE: THE TAX SERVICE SLOWS DOWN

Checkout runs a ten-hop synchronous chain. One day the tax service goes from 50 ms to 3 s. Every upstream service waits, worker threads fill up, and timeouts cascade until checkout fails, although nine of ten services are healthy. The fix: cache tax rates locally in pricing (they change rarely), move notifications to events, and cut the critical path to three hops, each with a tight timeout and a circuit breaker.

4 Key terms

Distributed monolith: separate services that are still tightly coupled

Nano-service: service too small to justify its overhead

Lock-step deploy: services that must be released together

Chatty interface: many small calls where one would do

Cascading failure: one slow dependency exhausts its callers

Service ownership: a named team accountable for a service

5 Interview questions

> **Q1. What is a distributed monolith?** BASIC
> Services that run separately but are tightly coupled: deployed together, changed together, sharing a database or business libraries, so you pay the network costs without independence.

Q2. Why is a shared database an anti-pattern? BASIC

It is a hidden contract: schema changes break other services, deployments couple, and one service's load slows others.

Q3. What signs show you have a distributed monolith? SCENARIO

Services that must deploy together, changes that ripple across services, shared databases or business-logic libraries, chatty calls, and long synchronous chains.

Q4. What's the fix? SCENARIO

Draw boundaries around business capabilities, give each service its own data, prefer events over long synchronous chains, merge services that always change together, and give every service an owning team.

Q5. A 10-hop chain at 99.9% per hop. What's the availability, and why is latency worse than the average suggests? TRICKY

About $0.999^{10} \approx 99.0\%$; and the slowest hop's tail latency hits most requests, since every request touches every hop.

Q6. Is merging services ever the right answer? TRICKY

Yes: if two services always change and deploy together or are too small to justify their overhead, one service (or a modular monolith) is better.

Next: SD-L-020 · Configuration Management backendroadmap.com